ノード価値に基づく準同型マークルツリーの証明更新の効率化

山本 竣介[1,a)]　野島 良[1]　宮地 秀至[1]

概要：ブロックチェーンの普及に伴い，大規模なデータの効率的な管理と検証が重要となっている．特に，Ethereum はブロックチェーン技術を用いた，多数のアカウント情報を管理するアカウントベースのシステムとして注目が集まっている．Ethereum のアカウント情報を効率的に管理・検証する仕組みとして，2013 年に Catalano らによってベクトルコミットメントが提案されて以来，より効率的な管理と検証を目的に様々な概念が適用され改良が行われてきた．その方式の 1 つとして，2023 年に Tas と Boneh は，複数のデータをツリー構造でコミットし，データの差分のみを用いて効率的な更新を可能とする準同型マークルツリーを用いた動的ベクトルコミットメントを提案した．彼らの方式では，各ノードについて，そのノードをルートとする部分木内の更新数が閾値を超える場合に，そのノードの値を更新情報として公開し，超えない場合はユーザが自身で計算する．しかし，この評価式は部分木内の更新数のみに基づいており，そのノードを公開することでユーザの計算をどれだけ削減できるかは反映されていない．本研究では，各ノードを公開した際に削減できるユーザの計算量をノード価値として定義し，更新数の代わりにノード価値を閾値と比較する方式を提案する．これにより，既存方式と同一の更新情報サイズのもとで，ユーザの証明更新時間をより多く削減することが可能となる．さらに，実際の Ethereum のデータを用いて既存方式との比較評価を行い，提案方式の有効性を示す．

キーワード：準同型マークルツリー，ベクトルコミットメント，ノード価値

Node-Value-Based Efficient Proof Updates for Homomorphic Merkle Trees

Shunsuke Yamamoto[1,a)]　Ryo Nojima[1]　Hideaki Miyaji[1]

Abstract: With the growing use of blockchain technology, efficient management and verification of large-scale data have become increasingly important. Ethereum manages account information using a tree-based data structure, for which vector commitments provide compact verification. In 2023, Tas and Boneh proposed a dynamic vector commitment based on homomorphic Merkle trees. Their method publishes a node value when the number of updated leaves in its subtree exceeds a threshold; otherwise, users compute the value themselves. However, this criterion considers only the number of updates and not the resulting reduction in user computation. We define the computational savings obtained by publishing a node as its “node value” and use this value for the publication decision. Experiments using real Ethereum data show that the proposed method reduces proof-update time while maintaining the same update information size as the existing method.

Keywords: Homomorphic Merkle Trees, Vector Commitments, Node Value

[1] 立命館大学
Ritsumeikan University
[a)] is0749xv@ed.ritsumei.ac.jp

1. はじめに

1.1 背景

ブロックチェーンは，取引記録を複数のノードで共有・

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

管理することにより，特定の管理者に依存せずにデータの改ざん耐性と透明性を実現する技術である [1]．近年では，ブロックチェーン上で管理されるデータの増加に伴い，大規模なデータを効率的に管理し，個々のデータの正当性を検証する仕組みが重要となっている．

Ethereum [2] は，ブロックチェーンに基づくアカウントベースの分散型システムであり，各アカウントの残高やスマートコントラクトの状態などを管理する．アカウント情報はツリー構造によって管理され，各アカウントの格納位置は，アドレスのハッシュ値に基づいて決定される．ツリーのルート値をブロックに記録することにより，アカウント情報全体の状態を短い値で表現できる．各ユーザは，自身のアカウント情報がツリーに正しく含まれていることを示す開示証明を保持することで，アカウント情報全体を保持することなく，自身のアカウント情報の正当性を検証できる．本論文では，アカウント情報全体を保持し，ツリーの構築，コミットメントの生成，開示証明の生成，および更新情報の生成を行う主体を**コミットメント生成者**と呼ぶ．これは特定の単一管理者を意味するものではなく，ベクトルコミットメント方式における役割を表す．Ethereum においては，ブロックに含まれるトランザクションを処理し，更新後の状態ツリーとそのルート値を計算するノードがこの役割に対応する．

Ethereum では，複数のトランザクションがブロック単位で処理され，ブロックが生成されるたびに多数のアカウント情報が更新される．このため，コミットメントだけでなく各ユーザの開示証明も，更新後のツリーに対して有効となるよう効率的に更新する必要がある．

1.2 ベクトルコミットメントと既存方式 TB23

ベクトルコミットメントは，複数のデータからなる順序付きベクトルに対して，短いコミットメント値を生成し，各位置 i のメッセージ m_i がコミットされたベクトルの i 番目の要素であることを示す開示証明によって検証できる暗号技術である [3]．動的ベクトルコミットメントでは，メッセージが更新された際に，コミットメント生成者が配布する更新情報を用いて，各ユーザが自身の開示証明を更新できる．

関連する技術としては，集合への帰属を証明するアキュムレータ [4] が挙げられる．また，ベクトルコミットメントの提案以来，証明を集約可能にした方式 [5] [6]，証明の更新・保守を効率化した式 [7] [8]，格子仮定に基づく方式 [9]，およびマークルツリーを多分木へ一般化した Verkle ツリー [10] [11] など，様々な方向性を持った研究が進められてきた．しかし，これらの動的ベクトルコミットメントでは，コミットメント生成者が配布する更新情報のサイズと，各ユーザが開示証明を更新するための計算量との間にトレードオフが存在する．例えば，代数的なコミットメント方式では更新情報を小さくできる一方，ユーザの証明更新計算量が更新数に比例して増加する場合がある [12]．一方，通常のマークルツリーではユーザの証明更新計算量を小さくできるが，更新情報サイズが更新数に比例して増加する [13]．

Tas と Boneh は 2023 年，更新情報サイズと証明更新時間のトレードオフを改善するため，準同型マークルツリーを用いた動的ベクトルコミットメント（以下，TB23）を提案した [14]．準同型マークルツリーは，通常のマークルツリーで用いられるハッシュ関数を，加算構造を持つ準同型ハッシュ関数に置き換えたツリー構造である [15]．この構造により，各内部ノードを，その部分木に含まれるメッセージの部分ダイジェストの和として表現できるため，メッセージが更新された場合には，更新前後の差分のみを用いて各ノードの値を更新できる．1 回の更新処理で変更されるメッセージ数を k，方式のトレードオフを制御するパラメータを $\nu \in (0,1)$ とし，ノード v をルートとする部分木に含まれる更新された葉の集合を $\mathcal{S}_v$ とする．TB23 では，各ノードについて，$|\mathcal{S}_v| > k^{1-\nu}$ を満たすノードについて更新後のノードの値を更新情報として公開し，満たさない場合には，各ユーザがメッセージの差分から更新後のノード値を計算する． この公開判定により，TB23 は更新情報サイズを $O\left(k^{\nu}\right)$ に抑えるとともに，ユーザの証明更新時間を $O\left(k^{1-\nu}\right)$ に抑えている．

1.3 本論文の貢献

TB23 のノード公開判定は，部分木内の更新数 $|\mathcal{S}_v|$ のみに基づいている．しかし，同じ更新数を持つノードであっても，ノードの深さや，そのノードを証明更新に利用するユーザ数によって，ノードを公開した際の計算削減効果は異なる．このため，更新数のみを用いた公開判定では，限られた更新情報サイズを必ずしも効果的に利用できない可能性がある．

そこで本研究では，各ノードを公開した際に削減できるユーザ全体の証明更新計算量を表す指標として，**ノード価値**を導入する．ノード v の深さを $d(v)$，ツリーの高さを h，ノード v の親ノードを $p(v)$ とする．本研究では，ノード価値を

$$\mathrm{value}(v) = \left|\mathcal{S}_{p(v)}\right| \cdot |\mathcal{S}_v| \cdot (h - d(v))$$

と定義する．ここで，$\left|\mathcal{S}_{p(v)}\right|$ はノード v を証明に含む更新ユーザ数を表し，$|\mathcal{S}_v|\,(h-d(v))$ は，各ユーザがノード v を自力で更新する際の計算量を表す．提案方式では，TB23 が用いる $|\mathcal{S}_v| > k^{1-\nu}$ という公開条件を，ノード価値に基づく $\mathrm{value}(v) > \lambda$ という条件に変更する．ここで，λ は更新情報サイズと証明更新時間のトレードオフを制御する閾値である．準同型マークルツリーの構成，再帰的な更新手順，および証明更新アルゴリズムは TB23 と同一であり，

© 2026 Information Processing Society of Japan

公開ノードを決定する評価式のみを変更する．

さらに，実際の Ethereum のブロックデータを用いて，既存方式 TB23 と提案方式を比較する．評価では，2 方式の更新情報サイズを同一に設定し，ユーザの平均証明更新時間および最大証明更新時間を比較する．その結果，提案方式は更新情報サイズを増加させることなく，平均証明更新時間を削減できることに加え，特に計算負荷が最大となるユーザの証明更新時間を大幅に削減できることを示す．

本論文の構成は以下のとおりである．第 2 章では，本論文でのベクトルコミットメントの定義を示し，第 3 章では，準同型マークルツリーを用いた既存方式 TB23 を説明する．第 4 章では，ノード価値に基づく提案方式を示し，第 5 章では，実際の Ethereum データを用いて既存方式との比較評価を行う．最後に，第 6 章で本論文をまとめる．

2. 準備

ベクトルコミットメント（Vector Commitment; VC）とは，順序付きメッセージ列に対して短いコミットメント値を生成し，あるメッセージがベクトル中の特定の位置に格納されていることを，開示証明を用いて検証できるコミットメント方式である [3]．動的ベクトルコミットメントは，コミットされたメッセージが変更された際に，コミットメントおよび開示証明を効率的に更新できるベクトルコミットメントである．

定義 2.1 (動的ベクトルコミットメント [14])**.** 動的ベクトルコミットメントは，以下のアルゴリズムによって構成される．ただし，$[N] = \{0, \ldots, N-1\}$ とする．

- **鍵生成** $\mathrm{KeyGen}(1^\lambda, N) \to pp$ ：セキュリティパラメータ λ とコミット対象となるベクトルのサイズ $N = \mathrm{poly}(\lambda)$ を入力として受け取り，公開パラメータ pp を出力する．公開パラメータ pp は，メッセージ空間 $\mathcal{M}$ を暗黙的に定める．
- **コミットメント生成** $\mathrm{Commit}_{pp}(m_0, \ldots, m_{N-1}) \to (C, \mathrm{data})$ ：メッセージ空間 $\mathcal{M}$ に属する N 個のメッセージからなるベクトル $\boldsymbol{m} = (m_0, \ldots, m_{N-1}) \in \mathcal{M}^N$ を入力として受け取り，コミットメント値 C と補助情報 data を出力する．data には，開示証明の生成およびコミットメントの更新に必要な現在のデータ構造の状態が含まれる．
- **開示** $\mathrm{Open}_{pp}(m_i, i, \mathrm{data}) \to \pi_i$ ：メッセージ m_i，位置 i，および補助情報 data を入力として受け取り，m_i がコミットされたベクトルの i 番目の要素であることを示す開示証明 π_i を出力する．
- **検証** $\mathrm{Verify}_{pp}(C, m, i, \pi_i) \to \{0, 1\}$ ：コミットメント値 C，メッセージ m，位置 i，および開示証明 π_i を入力として受け取り，証明を受理する場合は 1 を，拒否する場合は 0 を出力する．
- **コミットメント更新** $\mathrm{Update}_{pp}(C, (i, m_i, m_i')_{i \in I}, \mathrm{data}) \to (C', U, \mathrm{data}')$ ：更新前のコミットメント値 C，更新されたメッセージに関する情報，および更新前の補助情報 data を入力として受け取る．更新された位置の集合を $I \subseteq [N]$ とし，各 $i \in I$ について，位置 i のメッセージが m_i から m_i' へ更新されたものとする．ただし，$[N] = \{0, \ldots, N-1\}$ である．更新アルゴリズムは，更新後のコミットメント値 C'，開示証明の更新に用いる更新情報 U，および更新後の補助情報 data' を出力する．このとき，集合 I に含まれない位置のメッセージは変更されない．
- **証明更新** $\mathrm{ProofUpdate}_{pp}(C, p((i, m_i, m_i')_{i \in I}), \pi_j, m_j', j, U) \to \pi_j'$ ：位置 j に対応する更新前の開示証明 π_j を，更新後のコミットメント C' に対する開示証明 π_j' へ更新する．ここで，m_j' は更新後に位置 j に格納されるメッセージを表す．位置 j のメッセージが更新されていない場合は，$m_j' = m_j$ とする．出力された開示証明 π_j' は，更新後のコミットメント C' に対して $\mathrm{Verify}_{pp}(C', m_j', j, \pi_j') = 1$ を満たす．関数 $p(\cdot)$ は，開示証明の更新に必要なメッセージ更新情報の形式を表す．本稿で扱う *TB23* では，更新前後のメッセージ m_i および m_i' そのものを用いず，それらの差分のみを用いて証明を更新できる．したがって，$p((i, m_i, m_i')_{i \in I}) = (i, m_i' - m_i)_{i \in I}$ と定義できる．

正当性． 任意の $N = \mathrm{poly}(\lambda)$ に対し，正しく生成された公開パラメータ $pp \leftarrow \mathrm{KeyGen}(1^\lambda, N)$ のもとで，メッセージベクトル $(m_0, \ldots, m_{N-1}) \in \mathcal{M}^N$ に対するコミットメント C が Commit_{pp} によって生成され，必要に応じて更新されたものとする．また，位置 i に対する開示証明 π_i が Open_{pp} または $\mathrm{ProofUpdate}_{pp}$ によって正しく生成されたものとする．このとき，

$$\Pr\left[\mathrm{Verify}_{pp}(C, m_i, i, \pi_i) = 1\right] \geq 1 - \mathrm{negl}(\lambda)$$

が成立することを要求する．ここで，negl は無視可能関数である．

位置束縛性． VC の位置束縛性とは，任意の位置 $i \in [N]$ に対して，どのような確率的多項式時間攻撃者 $\mathcal{A}$ であっても，同一のコミットメントに対する異なる二つのメッセージを，同じ位置の正当な要素として受理させる確率が，セキュリティパラメータ λ に関して無視可能である性質をいう．

定義 2.2 (位置束縛性)**.** 任意の確率的多項式時間攻撃者 $\mathcal{A}$ に対し，$pp \leftarrow \mathrm{KeyGen}(1^\lambda, N)$, $(C, i, m, m', \pi_i, \pi_i') \leftarrow \mathcal{A}(pp)$ の結果,

$$\Pr\left[\begin{array}{c}\mathrm{Verify}_{pp}(C, m, i, \pi_i) = 1 \wedge \\ \mathrm{Verify}_{pp}(C, m', i, \pi_i') = 1 \ \wedge \ m \neq m'\end{array}\right] \leq \mathrm{negl}(\lambda).$$

が成立するとき，*VC* は位置束縛性を満たすという．

© 2026 Information Processing Society of Japan

また, 多くの VC は**秘匿性**も満たす．秘匿性とは, 任意の確率的多項式時間攻撃者 $\mathcal{A}$ が, コミットメント値がメッセージベクトル $(m_0, \ldots, m_{N-1})$ に対して生成されたものか, 異なるメッセージベクトル $(m'_0, \ldots, m'_{N-1}) \neq (m_0, \ldots, m_{N-1})$ に対して生成されたものかを識別できない性質をいう．本研究では，対象とする応用において秘匿性は明示的に必要とされないため考慮しない．また，VC は秘匿性を有するコミットメント方式と組み合わせることで秘匿性を付与できる [3].

3. 既存研究 TB23

Tas と Boneh は，準同型マークルツリーを用いた動的ベクトルコミットメント方式（以下，TB23）を提案した [14]. TB23 では，コミットメント生成者が配布する更新情報のサイズと，各ユーザが開示証明を更新するための計算量との間のトレードオフを制御できる．

本節では，後述する提案方式の理解に必要な TB23 の性質のみを説明し，準同型ハッシュ関数の具体的な構成は省略する．

3.1 準同型マークルツリー

準同型マークルツリーは，通常のマークルツリーのハッシュ関数を，格子仮定に基づく加算構造を持つハッシュ関数に置き換えられたツリー構造であり，Papamanthou らによって提案された [15]. 葉ノード数を $N = 2^h$ とし，h をツリーの高さとする．インデックスの集合を $[N] = \{0, \ldots, N-1\}$ と定義し，インデックス $i \in [N]$ の h ビットの 2 進表現を $\mathrm{bin}(i) = (b_1, \ldots, b_h)$ と表す．各ノードは，ルートからそのノードまでの経路を表すビット列 $v \in \{0,1\}^{\leq h}$ によって識別される．ルートノードは空のビット列 ε で表し，ノード v の深さを $d(v) = |v|$ とする．

ツリーの葉ノードには，コミット対象となるメッセージ $m_0, \ldots, m_{N-1}$ が対応する．具体的には，インデックス $i \in [N]$ に対応する葉ノードにメッセージ m_i を割り当てる．

ツリーの各ノード v にはノード値 u_v が付与される．葉ノードの値は対応するメッセージから計算され，各内部ノードの値は左右の子ノードの値から計算される．この処理を葉からルートに向かって繰り返すことで，ツリー全体のノード値が得られる．

メッセージ m_i が深さ d の祖先ノードの値に与える寄与を，部分ダイジェスト $h_{i,d}(m_i)$ と表す．各ユーザは公開パラメータとメッセージを用いて $h_{i,d}(m_i)$ を計算できる．ノード v をルートとする部分木に含まれる葉ノードのインデックス集合を

$$\mathcal{I}_v = \left\{ i \in [N] \;\middle|\; \begin{array}{c} \text{インデックス } i \text{ に対応する} \\ \text{葉ノードがノード } v \text{ をルート} \\ \text{とする部分木に含まれる} \end{array} \right\}$$

と定義する．準同型マークルツリーでは，ノード v の値 u_v は，その部分木に含まれる各メッセージの部分ダイジェストの和として $u_v = \sum_{i \in \mathcal{I}_v} h_{i,d(v)}(m_i)$ と表される．特に，ルートノードの部分木にはすべての葉ノードが含まれるため，$u_\varepsilon = \sum_{i \in [N]} h_{i,0}(m_i)$ となる．このルートノードの値をメッセージ列全体に対するコミットメントとし，$C = u_\varepsilon$ と定義する．

部分ダイジェスト関数 $h_{i,d}$ は線形性を持っており, メッセージ m_i が m'_i に更新された場合，祖先ノードに対する寄与の変化量は $h_{i,d}(m'_i) - h_{i,d}(m_i) = h_{i,d}(m'_i - m_i)$ と計算できる．この性質により，一部のメッセージが更新された場合でも，ツリー全体を最初から再構築する必要はなく，更新前のノード値に更新差分の寄与を加えることで，更新後のノード値を求められる．

3.2 更新情報の生成

1 回の更新処理において，更新されるインデックスの集合を $I \subseteq [N]$ とし，更新数を $k = |I|$ とする．各 $i \in I$ について，メッセージが m_i から m'_i に更新されたものとする．メッセージ m_i の更新前後の差分を $\Delta m_i = m'_i - m_i$ と定義し，1 回の更新処理における差分情報の集合を $D = \{(i, \Delta m_i) \mid i \in I\}$ とする．すなわち，D には，更新されたメッセージのインデックス i と，その更新前後の差分 Δm_i が含まれる．ノード v をルートとする部分木に含まれる更新インデックスの集合を

$$\mathcal{S}_v = \left\{ i \in I \;\middle|\; \begin{array}{c} \text{インデックス } i \text{ に対応する} \\ \text{葉ノードがノード } v \text{ をルートとする} \\ \text{部分木に含まれる} \end{array} \right\}$$

と定義する．ノード v に対する更新の影響を $\Delta u_v = \sum_{i \in \mathcal{S}_v} h_{i,d(v)}(\Delta m_i)$ とすると，更新後のノード値は $u'_v = u_v + \Delta u_v = u_v + \sum_{i \in \mathcal{S}_v} h_{i,d(v)}(\Delta m_i)$ と計算される．部分ダイジェスト関数は公開パラメータに含まれる関数を用いて計算できるため，各ユーザも差分情報 D から，必要な $h_{i,d(v)}(\Delta m_i)$ を計算できる．したがって，更新後のノード値が公開されていない場合でも，ユーザは更新前のノード値 u_v に，その部分木内で発生した更新の寄与を加えることで，更新後のノード値 u'_v を求められる．

TB23 では，パラメータ $\nu \in (0,1)$ を用い，各ノード v について

$$|\mathcal{S}_v| > k^{1-\nu} \tag{1}$$

を満たす場合に，更新後のノード値 u'_v を公開する．公開されるノード値の集合を

© 2026 Information Processing Society of Japan

$$U_{\text{node}} = \left\{ (v, u'_v) \;\middle|\; v \in \{0,1\}^{\leq h},\ |\mathcal{S}_v| > k^{1-\nu} \right\} \qquad (2)$$

と定義する．証明更新に用いる更新情報全体は，差分情報 D と，公開されるノード値の集合 U_{node} から構成され，$U = (D, U_{\text{node}})$ とする．パラメータ ν を大きくすると閾値 $k^{1-\nu}$ は小さくなり，より多くのノード値が公開される．この場合，更新情報のサイズは増加する一方で，各ユーザが自身で行う証明更新計算は減少する．反対に，ν を小さくすると公開されるノード数は減少するが，各ユーザの証明更新計算量は増加する．

3.3 開示証明の更新

インデックス i に対する開示証明 π_i には，i に対応する葉ノードからルートノードまでのパス上にある各内部ノードについて，その左右の子ノードの値が含まれる．これは，各内部ノードの値を左右の子ノードの値から再計算し，パスの正当性を検証するためである．

ツリーが更新されると，これらの子ノードの値も変化する可能性があるため，開示証明 π_i に含まれるノード値を，更新後のツリーに対応する値へ更新する必要がある．

証明更新に必要なノード v の更新後の値が U_{node} に含まれる場合，ユーザは公開されたノード値 u'_v を直接利用する．一方，U_{node} に含まれない場合，ユーザは開示証明 π_i に含まれる更新前のノード値 u_v を取り出し，差分情報 D を用いて

$$u'_v = u_v + \sum_{r \in \mathcal{S}_v} h_{r,d(v)}(\Delta m_r) \qquad (3)$$

を計算する．すなわち，ノード v の部分木内で更新されたメッセージの寄与のみを，更新前のノード値 u_v に加えることで，更新後のノード値 u'_v を求める．

1 つの更新メッセージから深さ $d(v)$ のノードに対する部分ダイジェストを計算するには，おおよそ $h-d(v)$ 回のハッシュ評価が必要となる．したがって，ノード v の更新後の値をユーザが自身で計算するための計算量は $|\mathcal{S}_v|\big(h-d(v)\big)$ に比例する．TB23 では，部分木内の更新数 $|\mathcal{S}_v|$ のみを用いて，ノード値を公開するか否かを判定する．しかし，同じ更新数を持つノードであっても，ノードの深さや，そのノード値を証明更新に必要とするユーザ数によって，ノードを公開した際に削減できる総計算量は異なる．

次章では，ノードを公開することで削減できるユーザ全体の証明更新計算量をノード価値として定義し，TB23 の公開判定を改良する．

4. 提案方式

4.1 提案方式の概要

本章では，既存方式 TB23 の効率を改善する方式を提案する．

4.2 評価式の改良

提案方式では，TB23 の更新アルゴリズムにおいて閾値と比較する評価式そのものを変更する．TB23 は各ノード v について，その部分木内の更新数 $|\mathcal{S}_v|$ を閾値 $k^{1-\nu}$ と比較し，$|\mathcal{S}_v| > k^{1-\nu}$ のときにノードの値を更新情報として公開する．これに対し提案方式では，$|\mathcal{S}_v|$ の代わりに，そのノードを公開したときにユーザの証明更新時間をどれだけ削減できるかを表すノード価値 $\text{value}(v)$ を用い，$\text{value}(v) > \lambda$ のときに公開する．提案はこの評価式であり，準同型マークルツリーの構成，再帰的な更新手順，証明更新の枠組みは TB23 と同一である．

4.2.1 記号の定義とノード価値の導出

記号は第 3 章と同じものを用い，新たに $\text{p}(v)$ をノード v の親ノードとし，$\mathcal{S}_{\text{p}(v)}$ を，親ノード $\text{p}(v)$ をルートとする部分木に含まれる更新葉ノードの集合とする．

ツリーの高さを h，ノード v の深さを $d(v)$ とする．ここで，ルートの深さを 0，葉の深さを h とすると，$h-d(v)$ は，ノード v から葉までの高さを表す．

$|\mathcal{S}_v| \geq 1$ であるノード v では，その部分木内のメッセージ更新に伴ってノード値も更新される．ノード v の更新後の値が更新情報に含まれていない場合，開示証明にノード v の値を必要とするユーザは，公開されたメッセージの差分から更新後のノード値を自身で計算する必要がある．このとき，ノード v の部分木内では，$|\mathcal{S}_v|$ 個の葉ノードが更新されている．各更新について，葉からノード v までの $h-d(v)$ 段階にわたって部分ダイジェストを計算するため，1 ユーザがノード v の更新後の値を計算するために必要なハッシュ評価回数は $|\mathcal{S}_v|\big(h-d(v)\big)$ となる．

一方，第 3.3 節で述べたように，準同型マークルツリーの開示証明には，パス上の各内部ノードについて，その左右両方の子ノード値が含まれる．したがって，ノード v の値は，親ノード $\text{p}(v)$ の部分木に含まれる葉に対応するユーザの開示証明で必要となる．本研究では，更新された葉に対応するユーザを評価対象とするため，ノード v の値を必要とするユーザ数を $|\mathcal{S}_{\text{p}(v)}|$ とする．ノード v の更新後の値を公開することで削減できるユーザ全体の計算量は，これら 2 つの積として表される．そこで，非ルートノード v のノード価値を

$$\text{value}(v) = |\mathcal{S}_{\text{p}(v)}| \cdot |\mathcal{S}_v| \cdot \big(h - d(v)\big) \qquad (4)$$

と定義する．

定理 4.1 (ノード価値の単調性)**.** 非ルートノード v の子ノードを w とする．このとき，$\text{value}(w) \leq \text{value}(v)$ が成立する．すなわち，ノード価値はルート側から葉側へ進むにつれて単調非増加である．

Proof. ノード w はノード v の子ノードであるため，$\text{p}(w) = v$ であり，ノード価値の定義より，$\text{value}(w) =$

© 2026 Information Processing Society of Japan

$|\mathcal{S}_v| \cdot |\mathcal{S}_w| \cdot \big(h - d(w)\big)$ となる．また，ノード w の部分木はノード v の部分木に含まれ，ノード v の部分木は親ノード $\mathrm{p}(v)$ の部分木に含まれるため，$|\mathcal{S}_w| \le |\mathcal{S}_v| \le |\mathcal{S}_{\mathrm{p}(v)}|$ が成立する．さらに，w は v の子ノードであることから，$d(w) = d(v)+1$ であり，$h-d(w) = h-d(v)-1 \le h-d(v)$ となる．以上より，$\mathrm{value}(w) = |\mathcal{S}_v| \cdot |\mathcal{S}_w| \cdot \big(h - d(w)\big) \le |\mathcal{S}_{\mathrm{p}(v)}| \cdot |\mathcal{S}_v| \cdot \big(h - d(v)\big) = \mathrm{value}(v)$ が成立する．∎

定理 4.1 より，あるノード v が $\mathrm{value}(v) \le \tau$ を満たす場合，v の任意の子孫ノード w についても $\mathrm{value}(w) \le \mathrm{value}(v) \le \tau$ が成立する．このため，ノード v が公開条件を満たさない場合，v の子孫について公開条件を確認する必要はなく，その時点で再帰処理を打ち切ることができる．一方，ノード v が $\mathrm{value}(v) > \tau$ を満たす場合には，v の更新後の値を公開し，その子ノードについて同じ判定を再帰的に行う．したがって，提案方式では，ルートノードから子ノードへ順に公開条件を判定する TB23 の再帰的な処理構造をそのまま利用できる．

4.2.2 閾値パラメータ τ とアルゴリズム

提案方式におけるノードの公開判定の閾値 τ は，TB23 におけるパラメータ ν と同様に，公開するノード数とユーザの証明更新計算量とのトレードオフを制御する．τ を小さくすると公開されるノード数が増加してユーザの証明更新計算量は減少し，τ を大きくすると公開されるノード数は減少してユーザの証明更新計算量は増加する．ただし，τ の適切な値は，ツリーの高さや更新分布によって変化するため，その値をあらかじめ直接指定することは難しい．そこで，提案方式では，目標とする公開ノード数 B に対し，上位 B 個が公開条件を満たすように τ を定める．

以上より，提案方式の更新アルゴリズムは次のようになる．コミットメント生成者はルートから再帰的に各ノード v を走査し，更新集合 $\mathcal{S}_v$ とその親の更新集合 $\mathcal{S}_{\mathrm{p}(v)}$ から式 (4) の $\mathrm{value}(v)$ を求め，$\mathrm{value}(v) > \tau$ を満たすときに限りそのノードを公開して両方の子へ再帰する．証明更新アルゴリズムは TB23 と同様に葉からルートへ各ノードを処理し，当該ノードが U に含まれれば取得し，含まれなければ自力で計算する．

5. 既存研究 TB23 との比較実験

本章では，TB23 の評価式をノード価値に変更した提案方式と既存方式 TB23 を実際の Ethereum データを用いて比較する．

5.1 実験環境と評価指標

実験は全て Python3.12（標準ライブラリ）で実装し，12th Gen Intel（R） Core（TM） i5-1245U （2.50 GHz）と 16GB のメモリを搭載した Windows 上で実験した．実験データには実際の Ethereum のブロックを用い，公開 RPC エンドポイント（`ethereum-rpc.publicnode.com`）から JSON-RPC によって取得した．各ブロックについて，含まれるトランザクションの送信元（from）および送信先（to）のアドレスを，そのブロックで更新されたアカウントとみなした[*1]．各アドレスは，その上位ビットによって高さ $h = 24$ のツリー（端数 $N = 2^{24}$，Ethereum の累計アカウント数と同程度）の葉に対応づけた．

提案方式は TB23 の評価式のみを変更したものであり，準同型ハッシュ関数そのものは 2 方式で共通である．そこで，本実験では格子ベースの準同型ハッシュ関数は実装せず，更新分布から公開ノード数とハッシュ評価回数を数え上げることににによって評価した．この評価方法は，Tas らが用いた解析的な指標と同一であり，TB23 について報告された値を再現できることを確認している．

更新情報サイズ $|U|$ は，コミットメント生成者がユーザに配布する更新情報の量（通信コスト）であり，$|U| =$ (公開ノード数) $\times |H|$，内部ノード 1 個のサイズは TB23 と同じ $|H| = 0.21$MB として，1 ブロックあたりの平均公開量 [MB/block] で示す．証明更新時間 T は，ユーザが更新情報 U を受け取った後，U に含まれないノードの値を差分から自力で計算するのに要する時間であり，この時間の計算に要するハッシュ評価回数に，1 回あたりの時間 $T_f = 2.74$ms を乗じて与える．ここで T_f は，安全なパラメータの下で TB23 が commodity hardware について報告した値である．T はハッシュ評価回数と T_f の積として算出されるため，本評価の結果はシミュレーションを実行した計算機の性能には依存しない．また，T にはユーザ自身のハッシュ計算時間のみが含まれ，更新情報 U の生成時間や送信・ダウンロード時間は含まれない．

k を 1 ブロックで更新されたアカウント数とする．2 方式は同一の $|U|$ に二分探索で揃えて比較し，T の平均と最大を報告する．「平均」は更新のあった葉に対応するユーザ全体にわたる平均値，「最大」はそれらのうち最も計算負荷の大きい 1 人の値である．また，図中の「提案/TB23」は，同一 $|U|$ における 2 方式の T の比であり，100%を下回るほど，提案方式が優れることを表す．なお，比較を公正に行うため，実験に用いる 1000 ブロックは実際の更新分布をそのまま用いる．

5.2 更新数 k への依存

1 ブロックあたりの更新数 k が異なる 4 つの 1000 ブロック（$k \approx 258, 409, 580, 632$）を別々に収集し，更新情報サイズは Tas らの実験と同様に $|U| = 2.1$MB/block に固定して比較した．結果を **表 1** に，2 方式の比を **図 1** に示す．

[*1] スマートコントラクト実行によるストレージの更新は from/to のアドレスには直接現れないが，それらを厳密に取得するには `debug_taceTransaction` などの高度な API が必要となるため，本実験では from/to に基づく近似を用いた．

© 2026 Information Processing Society of Japan

表 1: 異なる更新数 k での T（平均（ave）/最大（max））
Table 1 T(average (ave)/maximum (max)) for Different Numbers of Updates k

k	TB23 (ave/max)[s]	提案 (ave/max)[s]
258	6.13 / 19.64	5.76 / 13.60
409	9.34 / 25.42	8.80 / 14.54
580	14.51 / 48.36	12.09 / 22.35
632	16.32 / 52.53	13.17 / 21.16

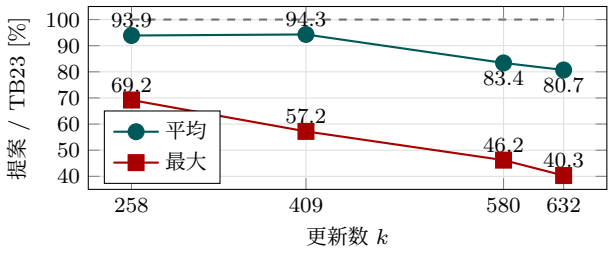


図 1: 異なる更新数 k における T の比（提案/TB23）
Fig. 1 Proportion of T at Different Numbers of Updates k (Proposal/TB23)

提案方式の T は平均・最大のいずれも更新数 k の全範囲で TB23 を下回り，の比は k が大きいほど小さくなる．k が大きいほど比が低下する要因は，公開すべきノードの選択の余地が広がるためと考えられる．k が小さい場合，公開する価値のあるノードは限られており，TB23 の更新数 $|\mathcal{S}_v|$ で選んでも提案方式のノード価値で選んでも，公開集合に大きな差は生じない．これに対し k が大きい場合には，同程度の更新数を持つノードが多数存在するようになる．このとき，TB23 の評価式では更新数のみで順序が定まるため，それらの中でどれを優先すべきかを区別できない．一方，提案方式はノード価値によって同じ更新数であってもより多くのユーザに関わるノードや葉から高い位置にあるノードを優先的に選択できる．この選択の差が，k の増加に伴う比の低下として現れたと考えられる．

平均よりも最大の比の低下が顕著である点は，ノード価値の定義から説明できる．因子 $|\mathcal{S}_{\mathrm{p}(v)}|$ は，そのノードを証明に含む更新ユーザ数を表し，更新が集中する領域のノードほど大きくなるため，提案方式ではそのようなノードが優先的に公開される．最も計算負荷の大きいユーザは，まさにその領域に自身の葉を持つユーザであり，提案方式の公開選択はこのユーザの負担を優先的に軽減する方向に働く．$k \approx 632$ において最大の T が 52.53 秒から 21.16 秒へと短縮されていることは，この効果を反映していると考えられる．Ethereum において更新数が多い状況は，更新情報サイズと証明更新時間がともに増大する最も負荷の高い局面であり，提案方式の利得はそこで最大となる．

5.3 更新情報サイズ $|U|$ への依存

前節を踏まえ，更新が多い環境（$k \approx 632$ の 1000 ブロック）を対象に，$|U|$ を変えて比較した．結果を **表 2** に，2 方式の比を **図 2** に示す．

表 2: $k \approx 632$ における $|U|$ ごとの T
Table 2 T for each $|U|$ at $k \approx 632$

$\lvert U\rvert$(MB)	TB23(ave/max)	提案 (ave/max)
2.1	16.32 / 52.53	13.17 / 21.16
4.2	7.88 / 23.12	6.77 / 12.66
8.4	3.84 / 10.13	3.47 / 7.22
14.7	2.21 / 5.41	2.06 / 5.42
25.2	1.35 / 3.38	1.28 / 3.61

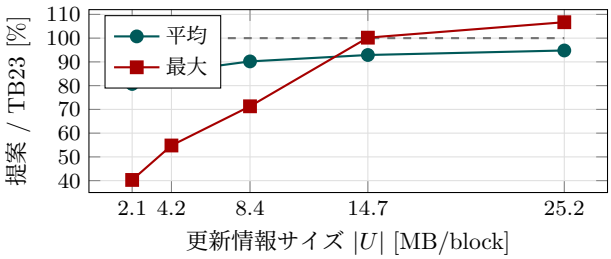


図 2: $k \approx 632$ における更新情報サイズ $|U|$ ごとの T の比（提案/TB23）
Fig. 2 Ratio of T per update size $|U|$ at $k \approx 632$ (Proposal/TB23)

平均は測定した全範囲で 100%を下回る一方，最大は $|U|$ の増加に伴って上昇し，$|U| \geq 14.7$MB/block では TB23 がわずかに上回る．$|U|$ が大きいほど比が 100%に近づく要因は，2 方式が公開するノードがほぼ同じになるためと考えられる．$|U|$ が小さい場合は公開できるノード数が限られるため，どのノードを選ぶか T を大きく左右するのに対し，$|U|$ が大きい場合は価値の高いノードは 2 方式ともに公開されるようになる．すなわち提案方式の優位は，公開するノードを選別する必要がある状況こそ生じる．

最大の比について $|U|$ が大きい領域で TB23 が上回る点は，ノード価値がユーザ全体の削減量を表す指標であり，特定のユーザの負担を直接抑えるものではないためと考えられる．公開ノード数に余裕がある状況では，TB23 の評価式でも更新が集中する領域のノードは十分に公開されるため，全体の削減量を優先する提案方式が TB23 に対して優位を保つ余地は小さい．ただし，その差は $|U| = 25.2$MB/block において 3.38 秒に対し 3.61 秒とわずかである．

以上より，提案方式は $|U|$ を増やすことなく T の平均を削減しており，とくに更新数が多く $|U|$ が限られる状況において有効であるといえる．

6. まとめ

本論文では，準同型マークルツリーを用いた動的ベクトルコミットメントである TB23 の証明更新を効率化する方式を提案した．

Ethereum のようなアカウントベースのシステムでは，

© 2026 Information Processing Society of Japan

ブロックの生成に伴って多数のアカウント情報が更新されるため，各ユーザが保持する開示証明を効率的に更新することが重要な課題となる．

TB23 は，準同型マークルツリーの加算構造を利用することで，更新情報サイズと証明更新時間のトレードオフをパラメータ ν によって制御する方式である．しかし，そのノード公開判定は部分木内の更新数 $|\mathcal{S}_v|$ のみに基づいており，同じ更新数を持つノードであっても，ノードの深さやそのノードを証明更新に必要とするユーザ数によって公開の効果が異なる点は考慮されていない．

この知見を踏まえ，本論文では，閾値を動的に変更するのではなく，公開判定に用いる評価式そのものを変更する方式を提案した．具体的には，ノードを公開した際に削減できるユーザ全体の証明更新計算量を表す指標として，ノードを証明に必要とするユーザ数 $|\mathcal{S}_{p(v)}|$，部分木内の更新数 $|\mathcal{S}_v|$，および葉からの高さ $h - d(v)$ の積からなるノード価値 $\mathrm{value}(v)$ を導入し，TB23 の公開条件をノード価値に基づく条件へ変更した．ノード価値が親ノードから子ノードへと減少していくことを示し，これにより TB23 の再帰的な処理構造をそのまま利用できることを明らかにした．提案方式が変更するのは評価式のみであり，準同型マークルツリーの構成，更新手順，および証明更新アルゴリズムは TB23 と同一である．さらに，実際の Ethereum のブロックデータを用いて既存方式 TB23 との比較評価を行った．2 方式の更新情報サイズを同一に設定した上で証明更新時間を比較した結果，提案方式は更新情報サイズを増加させることなく，証明更新時間の平均を削減できることを確認した．とくに，1 ブロックあたりの更新数が多い状況ほど，最も計算負荷の大きいユーザの証明更新時間を大きく削減できることを示した．一方で，更新情報サイズが大きい領域では，最も計算負荷の大きいユーザについて TB23 がわずかに上回る場合があることも確認された．

今後の課題として，本論文で提案したノード価値以外の評価式の検討が挙げられる．とくに，最も計算負荷の大きいユーザの証明更新時間を抑えることを目的とする場合には，ノード価値の定義にユーザごとの負担の偏りを反映させるなど，評価式のさらなる検討が必要である．

謝辞　本研究は，JST 経済安全保障重要技術育成プログラム JPMJKP24U1 の支援を受けたものです．

参考文献

[1] Nakamoto, S. et al.: Bitcoin: A peer-to-peer electronic cash system (2008).

[2] Wood, G. et al.: Ethereum: A secure decentralised generalised transaction ledger, *Ethereum project yellow paper*, Vol. 151, No. 2014, pp. 1–32 (2014).

[3] Catalano, D. and Fiore, D.: Vector Commitments and Their Applications, *Public-Key Cryptography - PKC*, Lecture Notes in Computer Science, Springer, pp. 55–72 (online), DOI: 10.1007/978-3-642-36362-7_5 (2013).

[4] Benaloh, J. and De Mare, M.: One-way accumulators: A decentralized alternative to digital signatures, *Workshop on the Theory and Application Cryptographic Techniques, 274–285* (1993).

[5] Tomescu, A., Abraham, I., Buterin, V., Drake, J., Feist, D. and Khovratovich, D.: Aggregatable subvector commitments for stateless cryptocurrencies, *SCN, 45–64, 2020.*

[6] Papamanthou, C., Srinivasan, S., Gailly, N., Hishon-Rezaizadeh, I., Salumets, A. and Golemac, S.: Reckle trees: Updatable merkle batch proofs with applications, *CCS, 1538–1551, 2024.*

[7] Wang, W., Ulichney, A. and Papamanthou, C.: {BalanceProofs}: Maintainable vector commitments with fast aggregation, *USENIX Security Symposium*, pp. 4409–4426 (2023).

[8] Luo, Z., Jia, Y., Gracia, A. V. O. and Kate, A.: Cauchyproofs: Batch-updatable vector commitment with easy aggregation and application to stateless blockchains, *IEEE S&P*, pp. 1947–1963 (2025).

[9] Peikert, C., Pepin, Z. and Sharp, C.: Vector and functional commitments from lattices, *Theory of Cryptography Conference*, Springer, pp. 480–511 (2021).

[10] Kuszmaul, J.: Verkle trees, *Verkle trees*, Vol. 1, No. 1, pp. 1–10 (2019).

[11] Feist, D.: Verkle Trie for Eth1 State, https://dankradfeist.de/ethereum/2021/06/18/verkle-trie-for-eth1.html (2021). Accessed: 2026-08-11.

[12] Kate, A., Zaverucha, G. M. and Goldberg, I.: Constant-size commitments to polynomials and their applications, *ASIACRYPT*, pp. 177–194 (2010).

[13] Merkle, R. C.: A digital signature based on a conventional encryption function, *Conference on the theory and application of cryptographic techniques*, Springer, pp. 369–378 (1987).

[14] Tas, E. N. and Boneh, D.: Vector Commitments with Efficient Updates, *AFT*, LIPIcs, Schloss Dagstuhl - Leibniz-Zentrum für Informatik, pp. 29:1–29:23 (online), DOI: 10.4230/LIPICS.AFT.2023.29 (2023).

[15] Papamanthou, C., Shi, E., Tamassia, R. and Yi, K.: Streaming authenticated data structures, *Annual international conference on the theory and applications of cryptographic techniques*, Springer, pp. 353–370 (2013).

© 2026 Information Processing Society of Japan